

FATEC – PROFESSOR ANTONIO SEABRA

LUÍS MIGUEL SCHIAZZA

IGOR RODRIGUES DOS SANTOS

Paulo Alvarez

Samuel Robles

RELATÓRIO TÉCNICO – Framworks e Suas Funções

PIRAJUÍ – SP

2026

Plataforma de Desenvolvimento Web

O desenvolvimento web moderno é dividido em duas grandes áreas: Front-End e Back-End. O Front-End é responsável pela interface visual e pela interação direta com o usuário, enquanto o Back-End é responsável pelo processamento das informações, regras de negócio, autenticação, integração com banco de dados e funcionamento interno do sistema. Atualmente, o mercado utiliza diversas linguagens, frameworks, bibliotecas e ferramentas que auxiliam no desenvolvimento de aplicações mais rápidas, organizadas, seguras e escaláveis.

Além disso, o desenvolvimento moderno também envolve integração com APIs REST, microsserviços, computação em nuvem, containers, DevOps e arquiteturas escaláveis, tornando o ecossistema web cada vez mais amplo e tecnológico.

Front-End

O Front-End corresponde à camada visual da aplicação, ou seja, tudo aquilo que o usuário consegue visualizar e interagir dentro do navegador ou dispositivo.

Linguagens Utilizadas

- **JavaScript (JS):** linguagem de programação utilizada para adicionar interatividade, manipulação de elementos, validações e dinamismo às páginas web.
 - **TypeScript (TS):** extensão do JavaScript que adiciona tipagem estática, orientação a objetos e maior organização para projetos de médio e grande porte.
 - **HTML5:** linguagem de marcação utilizada para estruturar páginas web, organizando conteúdos como textos, imagens, formulários, tabelas e mídias.
 - **CSS3:** linguagem responsável pela estilização visual, layouts responsivos, animações, efeitos visuais e adaptação para diferentes dispositivos.
-

Frameworks e Tecnologias Front-End

Os frameworks e bibliotecas front-end facilitam o desenvolvimento de interfaces modernas, reutilização de componentes e organização do código.

Frameworks CSS

- **Bootstrap:** framework CSS que disponibiliza componentes prontos, sistema de grid responsivo e estilização padronizada para acelerar o desenvolvimento.
- **Tailwind CSS:** framework utilitário que permite criar interfaces altamente personalizadas através de classes utilitárias diretamente no HTML.
- **Material UI:** biblioteca de componentes visuais baseada no Material Design da Google, muito utilizada em aplicações React.

Frameworks e Tecnologias JavaScript

- **Angular:** framework desenvolvido pela Google para construção de aplicações web robustas, escaláveis e organizadas utilizando TypeScript.
- **Next.js:** framework baseado em React que oferece renderização híbrida, SSR (Server Side Rendering), geração estática e otimizações de SEO.
- **Vue.js:** framework progressivo focado em simplicidade, flexibilidade e integração gradual em aplicações web.
- **Sass (SCSS):** pré-processador CSS que adiciona variáveis, funções, herança e modularização aos estilos.
- **Calendary.js:** biblioteca voltada para criação de calendários, agendas e gerenciamento de eventos em aplicações web.

Bibliotecas Front-End

As bibliotecas complementam funcionalidades específicas da aplicação.

- **React.js:** biblioteca JavaScript baseada em componentes reutilizáveis para construção de interfaces modernas e reativas.
- **jQuery:** biblioteca utilizada para simplificar manipulação do DOM, eventos, animações e requisições AJAX.
- **Axios:** biblioteca utilizada para realizar requisições HTTP entre front-end e back-end, principalmente em APIs REST.

Back-End

O Back-End é responsável pela lógica da aplicação, processamento das requisições, autenticação, gerenciamento de usuários, regras de negócio e comunicação com bancos de dados.

Linguagens Back-End

- **JavaScript/TypeScript:** amplamente utilizados com Node.js no desenvolvimento de APIs modernas e aplicações escaláveis.
- **PHP:** linguagem tradicional no desenvolvimento web, muito utilizada em sistemas administrativos e plataformas dinâmicas.
- **Python:** linguagem versátil aplicada em APIs, automações, inteligência artificial, ciência de dados e machine learning.
- **Java:** linguagem robusta e consolidada em sistemas corporativos, financeiros e aplicações empresariais.
- **Go (Golang):** linguagem desenvolvida pela Google com foco em performance, concorrência e aplicações distribuídas.
- **Kotlin:** linguagem moderna interoperável com Java, utilizada tanto no Android quanto no desenvolvimento back-end.
- **C#:** linguagem da Microsoft utilizada principalmente no ecossistema .NET para aplicações web e desktop.
- **Ruby:** linguagem focada em produtividade e simplicidade no desenvolvimento.

Frameworks Back-End

Os frameworks back-end ajudam na organização da arquitetura, criação de APIs e gerenciamento das funcionalidades do sistema.

JavaScript/TypeScript

- **NestJS:** framework modular baseado em TypeScript que utiliza arquitetura escalável e recursos avançados para APIs REST e microsserviços.
- **Express.js:** framework minimalista utilizado para criação de servidores HTTP e APIs em Node.js.
- **AdonisJS:** framework inspirado no Laravel que fornece estrutura completa para desenvolvimento em Node.js.

Java

- **Spring Boot:** framework empresarial utilizado para construção de APIs robustas, microsserviços e sistemas corporativos.

PHP

- **Laravel:** framework moderno que oferece ORM, autenticação, migrations e arquitetura organizada.
- **Symfony:** framework modular amplamente utilizado em aplicações corporativas e sistemas complexos.
- **Quarkus:** framework otimizado para ambientes cloud-native e microsserviços.
- **Micronaut:** framework leve focado em inicialização rápida e baixo consumo de memória.

Python

- **Django:** framework completo com autenticação, ORM, painel administrativo e estrutura pronta para aplicações robustas.
- **Flask:** microframework leve utilizado em APIs simples e pequenos serviços.
- **FastAPI:** framework moderno focado em alta performance, tipagem forte e documentação automática.

Ruby

- **Sinatra:** microframework minimalista utilizado para aplicações simples e APIs leves.

Kotlin

- **Ktor:** framework Kotlin utilizado para aplicações assíncronas e APIs modernas.

Banco de Dados

Os bancos de dados são responsáveis pelo armazenamento, consulta e gerenciamento das informações do sistema.

Bancos Relacionais

- **MySQL:** banco de dados relacional amplamente utilizado em aplicações web.
- **PostgreSQL:** banco relacional avançado conhecido pela estabilidade e recursos robustos.
- **MariaDB:** alternativa open source derivada do MySQL.
- **SQL Server:** banco de dados relacional da Microsoft utilizado em ambientes corporativos.

Bancos Não Relacionais (NoSQL)

- **MongoDB:** banco orientado a documentos muito utilizado em aplicações Node.js e arquiteturas escaláveis.
- **Redis:** banco em memória utilizado para cache, sessões e alta performance.
- **Firebase:** plataforma da Google que oferece banco em tempo real e diversos serviços integrados.

ORM (Mapeamento Objeto-Relacional)

Os ORMs permitem manipular bancos de dados utilizando classes e objetos ao invés de SQL puro.

- **Eloquent:** ORM padrão do Laravel utilizado em aplicações PHP.
- **TypeORM:** ORM para Node.js com suporte a TypeScript e múltiplos bancos relacionais.
- **Hibernate:** ORM amplamente utilizado em aplicações Java.
- **Prisma:** ORM moderno focado em produtividade, tipagem forte e segurança no acesso aos dados.

Ferramentas de Gerenciamento

As ferramentas de gerenciamento auxiliam na instalação, atualização e controle das dependências utilizadas pelos projetos.

- **NPM (Node Package Manager):** gerenciador de pacotes padrão do Node.js.

- **Composer:** gerenciador de dependências utilizado em aplicações PHP.
 - **Yarn:** alternativa ao NPM com foco em desempenho e gerenciamento otimizado.
-

Runtime

O runtime é o ambiente responsável pela execução do código da aplicação.

- **Node.js:** runtime baseado no motor V8 do Google Chrome utilizado para executar JavaScript no servidor.
 - **Deno:** runtime moderno criado pelo autor original do Node.js, focado em segurança e suporte nativo ao TypeScript.
-

DevOps e Infraestrutura

A área de DevOps e infraestrutura é responsável pela automação, deploy, integração contínua e gerenciamento dos ambientes da aplicação.

- **Docker:** plataforma utilizada para criação de containers, facilitando padronização e deploy das aplicações.
 - **Docker Compose:** ferramenta utilizada para orquestrar múltiplos containers simultaneamente.
 - **Git:** sistema de versionamento utilizado para controle e colaboração no código-fonte.
 - **GitHub/GitLab:** plataformas para hospedagem e gerenciamento de repositórios.
 - **NGINX:** servidor web utilizado como proxy reverso, balanceador de carga e servidor HTTP.
 - **Apache:** servidor web amplamente utilizado em aplicações PHP e hospedagens tradicionais.
-

Arquitetura e APIs

As aplicações modernas normalmente utilizam arquiteturas escaláveis e comunicação baseada em APIs.

- **API REST:** padrão de comunicação entre sistemas utilizando HTTP.
 - **JWT (JSON Web Token):** método de autenticação baseado em tokens.
 - **Microserviços:** arquitetura onde o sistema é dividido em pequenos serviços independentes.
 - **MVC (Model-View-Controller):** padrão arquitetural utilizado para separação de responsabilidades.
 - **SPA (Single Page Application):** aplicações que carregam uma única página e atualizam o conteúdo dinamicamente.
-

REFERÊNCIAS

ANGULAR. Angular Documentation. Disponível em: <https://angular.dev>. Acesso em: 05 maio 2026.

AXIOS. Axios Documentation. Disponível em: <https://axios-http.com/docs/intro>. Acesso em: 06 maio 2026.

BOOTSTRAP. Bootstrap Documentation. Disponível em: <https://getbootstrap.com/docs/>. Acesso em: 07 maio 2026.

COMPOSER. Composer Documentation. Disponível em: <https://getcomposer.org/doc/>. Acesso em: 08 maio 2026.

CSS. CSS: Cascading Style Sheets. MDN Web Docs. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/CSS>. Acesso em: 09 maio 2026.

DENO. Deno Documentation. Disponível em: <https://docs.deno.com>. Acesso em: 10 maio 2026.

DJANGO. Django Documentation. Disponível em: <https://docs.djangoproject.com>. Acesso em: 11 maio 2026.

DOCKER. Docker Documentation. Disponível em: <https://docs.docker.com>. Acesso em: 04 maio 2026.

EXPRESS. Express.js Documentation. Disponível em: <https://expressjs.com>. Acesso em: 05 maio 2026.

FASTAPI. FastAPI Documentation. Disponível em: <https://fastapi.tiangolo.com>. Acesso em: 06 maio 2026.

FLASK. Flask Documentation. Disponível em: <https://flask.palletsprojects.com>. Acesso em: 07 maio 2026.

GIT. Git Documentation. Disponível em: <https://git-scm.com/doc>. Acesso em: 08 maio 2026.

GO. Go Documentation. Disponível em: <https://go.dev/doc/>. Acesso em: 09 maio 2026.

HIBERNATE. Hibernate ORM Documentation. Disponível em: <https://hibernate.org/orm/documentation/>. Acesso em: 10 maio 2026.

HTML. HTML5 Documentation. MDN Web Docs. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTML>. Acesso em: 11 maio 2026.

JAVA. Java Documentation. Oracle. Disponível em: <https://docs.oracle.com/en/java/>. Acesso em: 04 maio 2026.

JAVASCRIPT. JavaScript Documentation. MDN Web Docs. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript>. Acesso em: 05 maio 2026.

JQUERY. jQuery API Documentation. Disponível em: <https://api.jquery.com>. Acesso em: 06 maio 2026.

KOTLIN. Kotlin Documentation. Disponível em: <https://kotlinlang.org/docs/home.html>. Acesso em: 07 maio 2026.

LARAVEL. Laravel Documentation. Disponível em: <https://laravel.com/docs>. Acesso em: 08 maio 2026.

MATERIAL UI. Material UI Documentation. Disponível em: <https://mui.com/material-ui/getting-started/>. Acesso em: 09 maio 2026.

MICRONAUT. Micronaut Documentation. Disponível em: <https://docs.micronaut.io>. Acesso em: 10 maio 2026.

NESTJS. NestJS Documentation. Disponível em: <https://docs.nestjs.com>. Acesso em: 11 maio 2026.

NEXT.JS. Next.js Documentation. Disponível em: <https://nextjs.org/docs>. Acesso em: 04 maio 2026.

NGINX. NGINX Documentation. Disponível em: <https://nginx.org/en/docs/>. Acesso em: 05 maio 2026.

NODE.JS. Node.js Documentation. Disponível em: <https://nodejs.org/docs/latest/api/>. Acesso em: 06 maio 2026.

NPM. NPM Documentation. Disponível em: <https://docs.npmjs.com>. Acesso em: 07 maio 2026.

PHP. PHP Documentation. Disponível em: <https://www.php.net/docs.php>. Acesso em: 08 maio 2026.

PRISMA. Prisma Documentation. Disponível em: <https://www.prisma.io/docs>. Acesso em: 09 maio 2026.

PYTHON. Python Documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: 10 maio 2026.

QUARKUS. Quarkus Documentation. Disponível em: <https://quarkus.io/guides/>. Acesso em: 11 maio 2026.

REACT. React Documentation. Disponível em: <https://react.dev>. Acesso em: 04 maio 2026.

RUBY. Ruby Documentation. Disponível em: <https://www.ruby-lang.org/en/documentation/>. Acesso em: 05 maio 2026.

SASS. Sass Documentation. Disponível em: <https://sass-lang.com/documentation/>. Acesso em: 06 maio 2026.

SPRING. Spring Boot Documentation. Disponível em: <https://spring.io/projects/spring-boot>. Acesso em: 07 maio 2026.

SYMFONY. Symfony Documentation. Disponível em: <https://symfony.com/doc/current/index.html>. Acesso em: 08 maio 2026.

TAILWIND CSS. Tailwind CSS Documentation. Disponível em: <https://tailwindcss.com/docs>. Acesso em: 09 maio 2026.

TYPESCRIPT. TypeScript Documentation. Disponível em: <https://www.typescriptlang.org/docs/>. Acesso em: 10 maio 2026.

TYPEORM. TypeORM Documentation. Disponível em: <https://typeorm.io>. Acesso em: 11 maio 2026.

VUE.JS. Vue.js Guide. Disponível em: <https://vuejs.org/guide/introduction.html>. Acesso em: 04 maio 2026.

YARN. Yarn Documentation. Disponível em: <https://yarnpkg.com/getting-started>.